

Reviewer Pack — Coxeter Matrix Entanglement Bounds Monotone Circuit Size

Entry 66af9a73b2dc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 05:04:54 UTC

Coxeter Matrix Entanglement Bounds Monotone Circuit Size

Entry ID: 66af9a73b2dc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 05:04:54 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Coxeter Groups) **Field B** (complexity object): Boolean Function Complexity (Monotone Circuit Size)

Statement:

[‘For any n , the size of a monotone circuit computing the function XOR_n is at least exponential in the entanglement entropy of the associated Coxeter matrix of order n .’, ‘Equivalently, for all $n \geq 3$ and for all monotone circuits C with size $s(C) = \Theta(2^n)$, there exists a Coxeter matrix W of order n such that the entanglement entropy $H(W)$ satisfies $H(W) = \Omega(2^{n/4})$.’]

Rationale (proposer’s reasoning):

[‘Coxeter groups have been studied for their connections to combinatorial optimization problems, including those related to circuit complexity.’, ‘Entanglement entropy measures quantum correlations and has been used in the study of communication complexity. This conjecture suggests a connection between these two areas.’, ‘If true, this would provide a new approach to proving lower bounds for monotone circuits, potentially revealing deeper structures in the complexity of Boolean functions.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dad1b9c569dee34a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \geq 3$ and monotone circuits C with $s(C) = \Theta(2^n)$, the associated Coxeter matrix entanglement entropy $H(W)$ satisfies $H(W) \geq 2^{n/4}$ for at least 80% of the seeds. The conjecture is falsified if there exists an $n \geq 3$ and a monotone circuit C with $s(C) = \Theta(2^n)$ such that the associated Coxeter matrix entanglement entropy $H(W) < 2^{n/4}$ for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter matrix" AND "monotone circuit size" - "entanglement entropy" AND "Boolean function complexity" AND "Coxeter group" - "XOR_n" AND "exponential bounds" AND "Coxeter matrix entanglement"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n):
        # Placeholder for actual circuit generation logic
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_entanglement_entropy(circuit):
        # Placeholder for actual entanglement entropy computation logic
        return sum(circuit) / len(circuit)

    n = random.randint(5, 40)
    circuit = generate_monotone_circuit(n)
    entropy = compute_entanglement_entropy(circuit)

    metric_name = 'entanglement_entropy'
    metric_value = entropy
    instances_tested = 1
    conjecture_holds = entropy >= 2**(n/4)
    counterexample = f'Circuit size {n} with entropy {entropy}' if not conjecture_holds else ''

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```

}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification under the pre-registered conditions. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11488
2	preregistration	ollama_remote	glm4:latest	0	0	6431
3	novelty	ollama_remote	glm4:latest	0	0	4745

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	5077
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41458
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13293
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7781
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6790
9	judge	ollama_remote	glm4:latest	0	0	19966

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117029 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/66af9a73b2dc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/66af9a73b2dc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/66af9a73b2dc.tar.gz` (if generated)